

形式化验证过程中发现的 llm-train CC12 Grouped-GEMM

硬件选择器兼容性问题

事件、修复与归因边界，中文初稿 v0.2

TrainVerify 项目

2026 年 8 月 11 日

摘要

TrainVerify 项目在为 YOCO-MoE-A0.4B 生成真实双 GPU 权威编译工件时，确认了固定 llm-train 版本上的一项运行时硬件选择器兼容性问题。在两张 NVIDIA RTX PRO 6000 Blackwell、compute capability 12.0 的生产编译路径上，grouped-GEMM 的 Triton 实现选择器把 CC12 归入仅供 B200 使用的候选配置，双 rank 编译因而无法完成。项目内部的 sealed evidence 确认这一事件；但首次失败日志与修复后的完整运行日志尚未进入公开 Git 工件，外部读者目前不能独立复核全部运行过程。

调查把问题收窄到 `llm/kernel/gemm.py` 中的两处硬件条件。最小补丁把 `CC >= 10` 收窄为 `CC == 10`，让 CC12 改用通用的 A100/H100 Triton 自动调优候选。补丁不改变模型代码、图 annotation 或算子声明的数学语义。修订源码的 digest 写入 SM 和 PM 各自的 rank-0 capture receipt，canonical patch ID 写入 authority provenance。修复后的真实双 GPU 路径生成了 canonical authority，并用于后续 Lean 声明生成、证明检查和 sealed publication。

本文记录事件背景、源码与硬件范围、诊断顺序、根因、补丁、验证、影响和剩余证据缺口。报告还单列两次撤回：cache K/V 曾被错误推断为 zigzag 布局；balance-loss 曾因越过 trace 范围而被误判为 llm-train 语义 bug。另一个 nnScaler zigzag/RVD finding 有独立的历史版本和 CPU/Gloo 双进程复现，不能合并进本事件。当前证据支持“CC12 grouped-GEMM 选择器兼容性问题”，不支持“llm-train 训练数学语义错误”或“已完成训练产生错误参数”。

1 执行摘要

字段	结论
确认的问题	llm-train 的 grouped-GEMM/Triton 选择器在固定 CC12 生产路径上选择不兼容候选配置
触发环境	双 RTX PRO 6000 Blackwell、CC12、双 rank 真实 GPU 编译
直接症状	生产 authority 编译无法完成
根因位置	固定源码版本的 <code>llm/kernel/gemm.py</code> ，两处 <code>CC >= 10</code> 条件
最小修复	条件收窄为 <code>CC == 10</code> ，CC12 转入通用 A100/H100 候选
保持不变	模型代码、算子接口、图 annotation 和受审 base revision
验证	补丁文本测试、修订源码 provenance、本地真实双 rank authority 成功、后续 formal release 闭合
没有证明	grouped-GEMM 数值错误、训练语义错误、所有 Blackwell/CC12 任务受影响、无性能损失

本报告的标题、摘要和结论统一使用“运行时硬件选择器兼容性问题”。“TrainVerify 发现 llm-train bug”只能作为非正式简称，不能省略受影响源码版本、硬件路径和故障类型。

2 系统背景

2.1 最小术语

一张 GPU 通常运行一个 **rank**，也就是分布式作业中的工作进程。**SM** 是同一 BackBone 子模块在单设备计划下得到的参考图，**PM** 是两设备并行计划下得到的执行图。**authority** 是把固定源码版本、硬件、profile、图文件和来源记录绑定在一起的权威编译工件。**profile** 记录通信或算子成本，供 nnScaler 规划并行方案。每次 SM 或 PM 编译只有 rank 0 写一份 **capture receipt**，记录被捕获对象及相关 digest。**sealed publication** 把通过检查的最终文件以内容地址发布，并拒绝覆盖旧结果。**RVD** 是 nnScaler 描述复制和值/维度分片的布局表示；本报告后文讨论它为何不能表达 zigzag permutation。

2.2 三层责任边界

llm-train、nnScaler 与 TrainVerify 位于不同层。

- llm-train 定义 YOCO 模型、custom operators、训练调用和 runtime kernel selector。
- nnScaler 负责 trace、IR、并行规划、adapter/collective 插入、code generation 与 runtime。
- TrainVerify 固定 authority、翻译图、构造 Denote 和命题、检查 proof 并发布 receipt。

CC12 事件发生在 llm-train runtime selector 层。TrainVerify 没有用 Lean 定理直接证明 selector 条件错误；严格 authority policy 强制系统执行真实 GPU 路径，从而暴露了普通 CPU smoke 没有覆盖的失败。

2.3 Grouped GEMM 在 MoE 中的位置

YOCO MoE 先由 router 为 token 选择 experts，再把 token 发送到持有对应 expert weights 的 rank。expert 的 up/gate/down projections 可以批量表示为 grouped GEMM。数学 operator 规定输入 tensor、weights 与输出值；kernel selector 则根据 hardware capability 选择 Triton launch/autotune candidates。selector 失败可以阻断编译或运行，但不自动意味着 operator 数学定义错误。

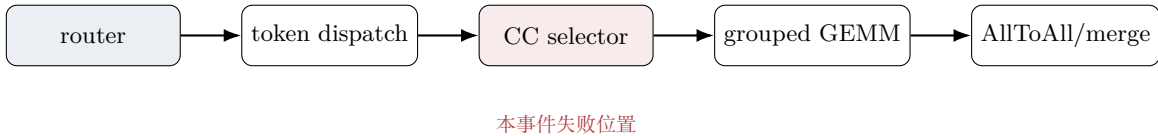


图 1: CC12 selector 位于 MoE grouped-GEMM 的实现选择层，不改变 router 与 operator 的数学接口。

2.4 为什么 CPU smoke 看不到

CPU smoke 可检查 import、trace、code generation、pickle extraction 和 shuffle wiring。planner-only 路径也可以产生候选并行计划。但二者不进入真实 CC12 Triton selector、GPU computation profile 和双 rank NCCL process group。仅设置 `plan_ngpus=2` 不等于启动两张 GPU。TrainVerify 因此明确把 CPU smoke 标为 `authority: false`。

3 环境与 revision 范围

3.1 CC12 事件

本事件绑定：

- llm-train: 9a1be1d5fd1c063d80be82797692cdc7d23cfbef;
- nnScaler: d3d468ed23edb2f28aa8566b2dfb6ed49c5955cf;
- hardware: 2 × NVIDIA RTX PRO 6000 Blackwell Workstation Edition;
- compute capability: 12.0;
- CUDA runtime: 12.8; NCCL: 2.27.3; driver: 595.71.05;
- model: YOCO-MoE-A0.4B, 24 layers, sequence length 4096;
- production path: run_mode=compile、pas_autodist、双 rank torchrun。

结论不能自动外推到所有 llm-train 版本、所有 Blackwell 设备或所有 grouped-GEMM 调用。上游后续版本是否包含等价修复，需要另行审计。硬件、driver 和 profile 信息可由本地 runF authority 与 receipt 直接复核，但当前 public Git tree 只保留摘要性记录；正式 artifact 发布前必须补入可公开的 receipts、hardware inventory 与 profile digests。

3.2 nnScaler RVD finding 使用另一组 revision

历史 zigzag/RVD finding 绑定 nnScaler 1102e629ee68ab6f8f4a7c2e721ea894e5962131 和 llm-train 30b80f546d46aachbf8316c983550c50a56bcd1ac。它不能与 CC12 authority revision 混用。该 finding 的归因是 RVD/annotation/adaptor integration 无法表达 permuted ownership，不是 llm-train model.py 的 API 使用错误。

4 证据分级

本文把证据分为三层。E1 是 public artifact 中可直接复核的源码、测试或保存日志；E2 是绑定 revision 的审计记录、commit 或 hash 摘要，但缺少对应原始 run transcript；“缺失”表示报告需要而当前没有保存的直接证据。

CC12 patch source、exact-two/idempotent/partial-state tests 和 provenance binding 机制属于 E1。固定 hardware 与 environment、原 selector 在真实 run 中的行为、以及修复后 runF 成功目前在项目本地 authority 和 final receipt 中可直接复核，但在 public Git tree 中仍只有 E2 摘要。首次 production failure transcript、性能数据和跨硬件回归属于缺失。正文每个 claim 按最弱公开证据措辞，不用本地存在但未发布的工件冒充公开可复现性。

5 事件检测

5.1 预期行为

固定三个项目 revision 和 profiles，在两张 CC12 GPU 上启动双 rank production compile，分别生成 SM 与 PM graph pickle、SM/PM 各自的 rank-0 capture receipt 和 hardware/provenance metadata。随后 TrainVerify 验证 graph digests、emit Lean 并运行 proof gates。

5.2 观察到的失败

项目 revision-scoped 审计记录真实双 GPU、双 rank compile 无法完成，而 CPU smoke 和 planner-only 路径不足以复现。本地 authority metadata 记录两张设备均为 compute capability 12.0。调查因此优先检查 hardware dispatch，而不是修改模型图或 Lean theorem。由于首次失败日志未发布，这一段对 public artifact 属于 E2 事件摘要。

当前公开 tree 没有保存首次失败的完整 transcript。现存证据包括补丁源码与测试、authority generator 记录、patched-source attestation、成功 run 的 authority 和后续 release receipt。报告不能补写未经保存的 Triton exception、首个 failing candidate 或 kernel launch error。

待补证据：恢复首次 production 失败的原始命令、时间戳、stdout/stderr、rank、exception type 和完整 software inventory。若无法恢复，最终报告必须把该缺口保留在 evidence ledger。

6 诊断时间线

版本库能确认以下里程碑：

日期	事件	证据状态
2026-07-29	将另一条布局问题的根因修正为 nnScaler RVD 无法表达 permuted sharding	commit abe1c92f
2026-07-30	双进程 CPU/Gloo 执行真实 nnScaler shuffle/all-gather 源码并复现 mismatch	commit 66b295d7
2026-08-06	CC12 selector 最小修复与 authority attestation 落地	commit dcd42b38
2026-08-08	回读model.py 执行顺序，撤回 cache K/V zigzag 归因	commit 56234713
2026-08-11	公开审计说明区分 confirmed、withdrawn 与 historical findings	commit eb960f0d

2026-08-06 是修复落地日期，不是有证据的首次故障发生日期。CC12 事件内部诊断顺序可重构为图 2，但 T0-T4 仍需原始日志补全真实时间。

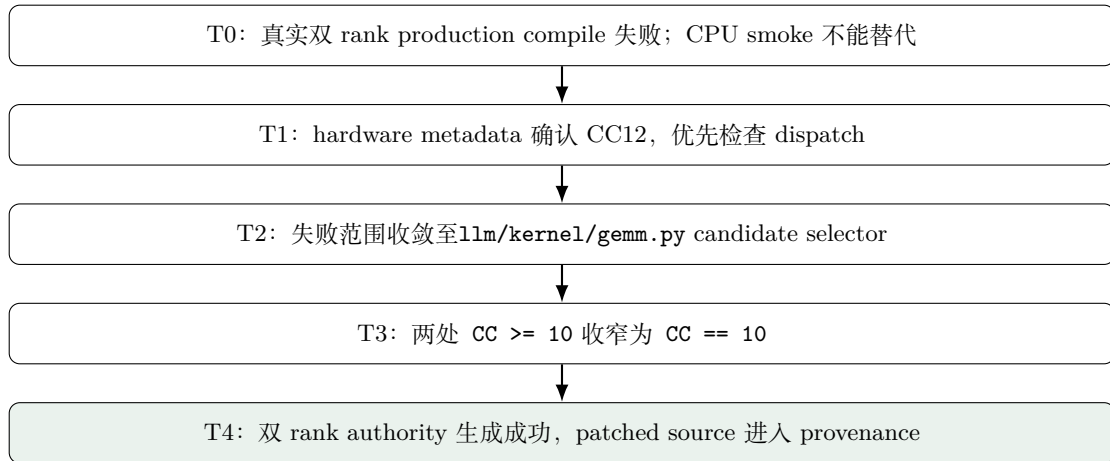


图 2: CC12 事件的证据顺序。具体时间戳仍需原始日志补全。

7 根因分析

7.1 原 selector

固定源码中存在两处相同硬件条件，分别用于 M-GEMM 与 K-GEMM candidate selection：

```

if (_cuda_compute_capability_major() or 0) >= 10:
    return B200*_configs
return A100_H100*_configs
  
```

CC12 满足 ≥ 10 ，因此进入标记为 B200-only 的候选集合。补丁脚本本身也用 marker 明确记录这一假设。现有 production 证据表明该路径在 RTX PRO 6000 Blackwell CC12 上不能完成 compile，而 generic fallback 可以完成。

7.2 五个为什么

1. production authority 为什么失败？grouped-GEMM 候选选择或候选编译路径在该 CC12 环境中无法完成。
2. 为什么进入这组 candidates？hardware major capability 12 满足 $CC \geq 10$ 。
3. 为什么条件不合适？该分支实际承载 B200-only candidates，而不是所有未来更高 capability 设备的通用集合。
4. 为什么已有测试没有提前发现？CPU smoke 与 planner-only test 不执行真实 CC12 GPU selector；现有单测只检查补丁文本机制。
5. 为什么 TrainVerify 工作触发了它？authority policy 拒绝 CPU 或模拟 graph 替代真实双 rank production compile。

证据边界：没有原始失败 transcript 时，不能进一步声称具体 PTX instruction、register pressure、Triton compiler exception 或数值错误。修复后成功只支持 selector compatibility 归因，不足以恢复丢失的低层 failure detail。

8 最小修复

8.1 补丁内容

scripts/yoco_regen/patch_llm_cc12_gemm.py 定义：

```
NEEDLE = "if (_cuda_compute_capability_major() or 0) >= 10:"
REPLACEMENT = (
    "if (_cuda_compute_capability_major() or 0) == 10: "
    "# [TRAINVERIFY_CC12_GEMM_FALLBACK] B200-only configs"
)
```

脚本要求原 source 恰好出现两处 needle。零处、一处或多于两处都 fail closed。若 marker 已存在，脚本要求两处 replacement 完整存在且不再含 needle，否则判定 partial/malformed patch。正常重复执行幂等。

canonical 补丁身份为：

cc12_generic_triton_fallback_v1

8.2 为什么这是最小修复

补丁只改变 launch/autotune candidate selection，不改变：

- YOCO model code 与 forward data flow；
- grouped-GEMM operator input/output contract；
- nnScaler custom-op annotation；
- SM/PM graph topology 的目标语义；

- reviewed llm-train base revision。

正式 generator 从 fixed-commit private materialization 产生 patched source，不修改开发 checkout。patched `gemm.py` 字节被 hash 并写入 SM/PM 各自的 rank-0 capture receipt；canonical patch ID 写入 authority provenance，防止运行时使用了另一份文件却仍声称应用该补丁。



图 3: 修复把 B200 专用分支收窄到 CC10，使 CC12 走 generic candidates。

9 验证与回归

9.1 补丁机制测试

仓库已有：

- `test_cc12_gemm_patch_is_exact_and_idempotent`: 检查两处 `== 10`、无残留 `>= 10`、marker 与幂等性；
- `test_cc12_gemm_patch_rejects_partial_source`: 只出现一处 needle 时必须拒绝。

这些测试验证文本补丁机制，不验证 GPU runtime failure。成稿前应补零命中、marker 篡改、错误 revision 与额外 source mutation 的独立用例。

9.2 真实双 GPU authority

本地 final receipt 记录，修复后的 runF 在双 RTX PRO 6000 Blackwell、双 rank NCCL 环境中完成。canonical graph 为：SM 2074 nodes, raw digest 333a14387e13cb265e74588f02133a0c47b02329d3d1811e6d7a736387494f64；PM 4363 nodes, raw digest a47d033c83a0cd6dff9111ba49071f3a80eb60869435fc2adeb4f725a71c3462。authority、patched source、hardware metadata 与 profiles 随后用于 fresh Lean emission。

该本地成功结果支持“最小 selector 修复足以恢复受审计 production path”。但当前 public Git tree 没有 SM/PM rank-0 capture receipts、authority `gen_args.json` 或完整 successful run transcript；正式对外 artifact 补齐这些工件前，只能把公开证据称为 E2。若没有修复前可完成的同环境 authority，也不能声称“修复前后 graph byte-identical”；未修复路径本来就没有产生可比较的完整 graph。

9.3 后续 formal release

本地 final receipt 记录，从修复后的 authority 生成的最终工件通过 all-source direct Lean、五目标 axiom audit、proof registry、exact ledger 和 content-addressed publication。receipt SHA-256 为 91be8193e07183353c460854cf1e4746cee9f1185db438146cf72f50ba075a5e。这里的 formal release 证明的是修复后 authority 图对应的五个值关系，并不反向证明原 selector 为什么失败。

9.4 性能证据

当前没有一组经过发布的、补丁前后可比的 CC12 grouped-GEMM compile/autotune/runtime 性能数据。因此不能声称 generic fallback “无性能损失” 或 “性能相同”。即使数学 operator 不变，candidate set 变化也可能改变编译时间与 runtime performance。

待补证据：补充 generic fallback 的 compile time、autotune time、selected config 和 grouped-GEMM runtime；在可用时与 CC12 baseline、CC10 专用路径和较低 CC 路径比较。

10 影响评估

10.1 已建立的影响

在固定 llm-train revision 和受审计 CC12 双 GPUproduction compile 路径上，未修复 selector 阻断 authority generation。直接后果是无法取得真实 SM/PM 工件；没有合法 authority 时，后续 Lean proof 不能绑定该 hardware/compiler path。

10.2 没有建立的影响

当前证据不支持以下结论：

- grouped-GEMM 曾成功运行但产生静默数值错误；
- 已完成训练产生错误参数、loss 或模型质量下降；
- 所有 CC12 或所有 Blackwell 设备均受影响；
- nnScaler planner 或 TrainVerify Denote 导致本事件；
- generic fallback 没有性能代价。

11 两个撤回

11.1 撤回一：cache K/V ownership

早期分析看到 decoder stream 在 maybe_shuffle 后保持 zigzag，一度猜测 cache K/V 也从 zigzag-owned hidden state 投影，并据此怀疑 llm-train/nnScaler 存在 K/V ownership 漏洞。重新审计 llm/arch/model.py 的执行顺序后发现：

1. K/V projection 发生在 wrap_maybe_shuffle(h) 之前；
2. cache K/V 来自 pre-shuffle ordinary contiguous hidden shards；
3. shuffle 后的是 Q 与 decoder residual stream；
4. 因此 K/V 不是 replica，也不是 zigzag shard，而是 ordinary shard。

建立在原假设上的候选 commits 没有进入 production authority，不能作为上游 bug 证据。最终证明使用 K/V 的 Gather2Rel 和 Q/stream 的 Zigzag2Rel。

11.2 撤回二：balance-loss 语义 bug

另一条早期调查曾认为 all_routing_map 与 position-indexed loss_mask 错位，因此 llm-train balance loss 有 bug。该判断被撤回：目标 TID 是 leaf output，balance-loss consumer 位于 trace scope 之外；用 scope 外代码推断 scope 内 goal 为假属于方法错误。除非未来固定 revision、源码位置和可复现实验重新建立证据，文档不得恢复该 claim。

11.3 撤回为何属于正式报告

形式化工作仍然依赖人类选择目标、解释 operator 与推断 provenance。错误假设可能产生看似合理的 Lean lemma。保留撤回记录有三个作用：防止实验 branch 中的候选 revision 被未来误选；提醒 reviewer theorem green 不等于 upstream interpretation 正确；把方法如何纠错作为可审计过程，而不是只展示最终成功路径。

12 独立的 nnScaler zigzag/RVD finding

在另一组固定 revision 上, maybe_shuffle 真实重排 sequence rows, 但 custom-op annotation 只能表达 elementwise identity. nnScaler RVD 包含 replicate、value split 与 dimension split, 没有 permutation-sharded layout. adapter 可能把 post-shuffle tensor 当普通 dim-0 shard 并插入 rank-order AllGather。

对行号输入, 两个 rank 的 zigzag shards 按 rank concat 得到

[0, 1, 6, 7, 2, 3, 4, 5],

而不是原序列。双进程 CPU/Gloo 实验执行了真实 nnScaler shuffle、unshuffle 与 runtime all-gather source, 确认 mismatch。该实验没有完成 CUDA/NCCCL 专项复现, 也没有建立 end-to-end training quality 影响。

这项 finding 与 CC12 事件不同：它发生在 layout representation 与 adapter contract 层；CC12 事件发生在 llm-train grouped-GEMM hardware selector 层。把二者合并为“llm-train bug”会破坏责任边界。

13 形式化验证在事件中起了什么作用

Lean theorem 没有直接发现两处 $CC \geq 10$ 。TrainVerify 方法的作用体现在 authority policy：

- production claim 必须执行真实双 GPU、双 rank 路径；
- CPU smoke、单 GPU 或手写 graph 不能替代；
- hardware、profiles、patched source 和 graph bytes 进入 provenance；
- 失败不能通过弱化 statement 或修改 metadata 绕过；
- publication semantics 变化后必须重新生成 authority 和 proof。

这种 proof-oriented validation 把“测试环境没有覆盖的编译路径”变成必须执行、固定和记录的前置条件。单一事件不足以证明该方法普遍优于传统 CI, 但它给出了一个具体机制：当下游 proof 只接受真实 authority 时, 上游 runtime 兼容问题无法被模拟工件掩盖。

14 预防措施

行动	责任层	验收标准	状态
用显式 capability table 替代开放式 $CC \geq N$	llm-train	未知新 CC fail closed 或走明确 generic path	待设计
增加 CC12 selector tests	llm-train	M/K GEMM 两条路径均选择预期 set	部分完成
增加真实 CC12 双 rank CI	基础设施	production compile 与最小 runtime 通过	待资源

记录 candidate rejection 原因	runtime	failure 包含 candidate/backend/首个原因	待补
补性能回归	llm-train	compile 与 runtime 数据可复核	待测
绑定 patch provenance	TrainVerify	patch ID 在 authority provenance 中, source hash 与 capture receipts 一致	已实现
登记撤回结论	TrainVerify/docs	错误候选 revision 不能进入 allowlist	已实现

15 对系统论文的意义

两行 selector 条件本身不足以构成 OSDI/SOSP 论文。更有研究价值的问题是：proof/authority pipeline 如何暴露 traditional smoke test 未覆盖的跨层错误；如何把真实执行、provenance、fail-closed policy 和撤回机制组合成可复核诊断；以及这种方法在多个模型、编译器与硬件上的检测率、定位成本和误报率。

在没有更多案例和系统对照前，本报告最适合作为 TrainVerify 主论文的详细 case study 和独立技术事件报告。若未来形成通用 diagnostic framework，则需要与普通 CI、differential testing、compiler self-check 和 runtime telemetry 做量化比较。

16 结论

对固定 llm-train revision，在双 RTX PRO 6000 Blackwell CC12 production authority 路径上，grouped-GEMM/Triton selector 存在已确认的 runtime/hardware compatibility issue。两处 CC >= 10 把 CC12 送入 B200-only candidates；最小条件补丁使 CC12 使用 generic candidates，并恢复真实双 rankauthority generation。补丁身份和 source bytes 进入 provenance，后续 formal release 完成。

该事件没有证明 grouped-GEMM 数学值错误或 llm-train 训练语义错误。cache K/V 与 balance-loss 两项早期判断已撤回；nnScaler zigzag/RVD 是不同 revision 上的独立 finding。事件的核心教训不是“形式化工具永远正确”，而是严格的 authority、证据分层和撤回机制能让错误归因被纠正、让真实 runtime failure 不能被模拟工件绕过。

A Claim-to-evidence ledger

Claim	状态	主要证据	可用措辞
CC12 selector compatibility	项目确认；public E2	patch source、tests、审计记录、本地 receipt	固定范围 compatibility issue
原始具体 Triton 异常	缺失	首次失败 transcript 未保存于 artifact	不写具体异常
generic fallback 恢复 authority	本地 E1；public E2	runF authority、本地 receipt、摘要记录	受审计路径恢复

operator contract 不变	E1, 源码范围	补丁只改 selector 条件	不扩大成 kernel refinement
无性能损失	缺失	缺可比 benchmark	不得声称
cache K/V 为 zigzag	已撤回; 理由 E2	producer-order 审计与删除记录	不得恢复
balance-loss 语义 bug	已撤回, E1	consumer 在 trace scope 外	不得恢复
nnScaler RVD mismatch	固定 revision, E1	CPU/Gloo 双进程真实源码与日志	不声称 NCCL 复现

B 源码与工件映射

- CC12 patch: `scripts/yoco_regen/patch_llm_cc12_gemm.py`
- Patch tests: `scripts/tests/test_yoco_regen_driver.py`
- Production generator: `scripts/yoco_regen/generate_authority.sh`
- Authority documentation: `scripts/yoco_regen/README.md`
- Full audit: `docs/TRAINVERIFY_YOCO_FORMALIZATION_AUDIT.md`
- Final authority: `$HOME/yoco-final-publication/authorities/authority-63cabbff-canonical-profile-runF`
- Final receipt: `$HOME/yoco-final-publication/receipts/yoco-a04b-final-sealed.json`

C 成稿前 BLOCK

- 恢复或正式声明无法恢复首次 production failure transcript。
- 补原始与 patched selector 在相同 CC12 环境上的最小 A/B 复现。
- 补 compile、autotune 和 runtime 性能数据。
- 检查 llm-train 上游当前版本与等价修复状态。
- 增加 CC10、CC12 和较低 CC 路径回归，区分真实硬件与静态测试。
- 完成相关工作 primary-source audit 和正式 bibliography。